

Binary Classification of Mortgage Loan Approval Using HMDA Data

ADS 2026 — Final Project Report | **Author:** Reza Kazemeiny Moghaddam | July 25, 2026

Abstract. This report presents a machine learning pipeline for predicting mortgage loan approval using HMDA data. Starting from a 15-million-row raw dataset, systematic cleaning and feature selection reduced it to 8 million rows, from which 500,000 were sampled. Four classifiers (Logistic Regression, Random Forest, CatBoost, LightGBM) were trained under three class-balancing strategies. CatBoost on the full class-weighted dataset achieved the best ROC-AUC. Several deep-learning architectures and a custom AdaBoost variant were explored but rejected due to weaker performance and higher training cost.

1 Problem Statement

Under the Home Mortgage Disclosure Act, U.S. financial institutions publicly report mortgage application outcomes. The task is to predict `loan_approved` $\in \{0, 1\}$ from features available at application time. The problem is non-trivial due to class imbalance ($\approx 4:1$), high-cardinality categoricals, skewed numeric distributions, and a decade of data spanning the 2008 financial crisis.

Beyond predictive accuracy, the problem has societal relevance: biased lending models can reinforce historical discrimination. Our pipeline deliberately removes explicit racial identifiers to reduce demographic dependence while retaining the ability to audit fairness post-hoc.

2 Dataset and Preprocessing

The raw HMDA files contain ~ 15 million rows and 70 columns. After cleaning, the dataset shrank to ~ 8 million rows (≈ 17 columns). Around 40 columns were removed based on four criteria: (1) columns with a high rate of missing or invalid values; (2) numeric-code columns that are direct encodings of a name-column already retained; (3) explicit racial/ethnic identifier columns, to reduce racial dependence in model decisions; (4) columns containing post-decision information that would cause target leakage. A **stratified 500,000-row sample** was drawn from the cleaned data (preserving the $\approx 80/20$ class split) to keep training times tractable.

3 Feature Engineering

Exploratory visualisation (histograms, pairplots, correlation heatmaps) showed that no single raw feature provides strong univariate discrimination — mutual information between individ-

ual columns and the target was low in isolation, and class distributions overlapped heavily in one dimension. Seventeen derived features were therefore constructed: financially motivated ratios (e.g. `loan_to_income`, `income_to_median_income`, `owner_occupied_share`) and log-transforms of all right-skewed numeric columns. A binary `has_co_applicant` flag was also added. A subsequent *random feature search* — pairing and combining existing columns randomly and measuring AUC impact — confirmed that the domain-motivated features captured most available signal; random combinations yielded only marginal and inconsistent gains.

4 Methodology

4.1 Class-imbalance strategies

Three training sets were derived from the 80% training split ($\approx 400k$ majority / $100k$ minority):

1. **Under-sampled (100k/100k):** majority class down-sampled to match minority size.
2. **200k/200k:** majority down-sampled to 200k; minority over-sampled with replacement to 200k.
3. **Full unbalanced + class weights:** all training rows kept; models use `class_weight='balanced'` to up-weight minority errors.

4.2 Models

Four model families were trained on each of the three datasets above, giving 12 model instances in total.

Logistic Regression serves as the interpretable linear baseline. Categorical features are one-hot encoded (`min_frequency=50` to group rare levels); numerics are median-imputed and standard-scaled. Solver: `saga`, `max_iter=1000`.

Random Forest uses `n_estimators=250` and `min_samples_leaf=20`. `max_depth` is selected from $\{4, 6, 8, 10, 12, 14, 16, 18, 20\}$ via an internal 80/20 validation fold, optimising ROC-AUC then balanced accuracy then F1.

CatBoost (`iterations=1200`, `learning_rate=0.05`, `depth=10`) receives raw categorical columns through its native `cat_features` interface, avoiding the memory overhead of OHE. Early stopping is applied (`od_wait=50`) monitored on a balanced evaluation set.

LightGBM (`n_estimators=700`, `learning_rate=0.04`, `num_leaves=64`, `subsample=0.85`, `colsample_bytree=0.85`) receives categorical columns as `pandas category` dtype.

A **majority-vote ensemble** was evaluated both within each model family (across the three training sets) and across all 12 instances (requiring ≥ 7 positive votes for a positive prediction).

Evaluation metrics: ROC-AUC (primary ranking metric), balanced accuracy, F1-score, precision, and recall, all computed on the held-out stratified 20% test set.

5 Results

5.1 Retained models

Table 1: Performance of retained classifiers (fill from artifacts/final_sorted_results.csv)

Model	Training set	Bal.Acc	Precision	Recall	F1	AUC.
Logistic Regression	under-sampled	0.7167	0.9086	0.7330	0.8114	0.7876
Logistic Regression	200k/200k	0.7185	0.9089	0.7373	0.8141	0.7885
Logistic Regression	unbalanced	0.7181	0.9091	0.7342	0.8123	0.7881
Random Forest	under-sampled	0.7030	0.9034	0.7177	0.7999	0.7715
Random Forest	200k/200k	0.7028	0.8999	0.7402	0.8123	0.7710
Random Forest	unbalanced	0.7026	0.9001	0.7378	0.8109	0.7703
CatBoost	under-sampled	0.7417	0.7284	0.7708	0.7490	0.8222
CatBoost	200k/200k	0.7291	0.7144	0.7635	0.7381	0.8017
CatBoost	unbalanced	0.7439	0.7196	0.7991	0.7573	0.8253
LightGBM	under-sampled	0.7262	0.7189	0.7431	0.7308	0.8034
LightGBM	200k/200k	0.7235	0.6916	0.8069	0.7448	0.7985
LightGBM	unbalanced	0.7284	0.6999	0.7997	0.7465	0.8054
All-Model Majority Vote	ensemble	0.7356	0.7269	0.7547	0.7405	0.8158

Bold row = best configuration by ROC-AUC. Majority-vote ensembling did not improve over the best individual model, indicating correlated errors across sampling strategies.

Key observations: class-weighted full training yielded the highest AUC; 200k/200k produced the best F1; under-sampling maximised recall at the cost of precision.

5.2 Deep-learning baselines

Six neural architectures were benchmarked for comparison.

Table 2: Deep-learning model results

Architecture	ROC-AUC	Avg.Precision	F1
MLP + Embedding	0.7925	0.9302	0.8202
Wide & Deep	0.7896	0.9291	0.8362
FT-Transformer	0.7992	0.9323	0.8255
AutoInt	0.7989	0.9326	0.8244
NODE	0.7714	0.9219	0.8041
TabNet	0.7527	0.9130	0.8390

All six were rejected: ROC-AUC was lower than the gradient-boosted models while training time per epoch was 5–20× higher, making hyperparameter search impractical.

6 Approaches That Did Not Work

6.1 Deep-learning architectures (MLP, Wide & Deep, FT-Transformer, AutoInt, NODE, TabNet)

All six neural architectures (Table ??) underperformed CatBoost and LightGBM on ROC-AUC while requiring 5–20 $\times$ more training time per epoch. The core issue is a well-documented property of tabular data: gradient-boosted trees tend to outperform neural networks when features are heterogeneous and predictive signal is spread across many individually weak predictors — which is precisely the structure revealed by the mutual information analysis in this dataset.

Among the neural approaches, **FT-Transformer** and **AutoInt** came closest (AUC ≈ 0.799), followed by **MLP + Embedding** (AUC 0.793) and **Wide & Deep** (AUC 0.790). **NODE** (AUC 0.771) and **TabNet** (AUC 0.753) performed worst; TabNet’s sparse feature-selection attention mechanism appears poorly suited to a setting where many features carry small additive contributions, causing it to prematurely discard informative columns.

None of the six architectures offered a compensating advantage in interpretability or convergence speed that would justify the performance gap.

6.2 Custom AdaBoost with random feature search

A bespoke AdaBoost variant was implemented in which each weak learner was trained on a dynamically selected feature subset. Features were chosen at each step by a compound criterion: high mutual information with the target combined with low correlation with features already used by earlier learners, promoting diversity across the ensemble. At each boosting round the algorithm attempted random feature additions and removals, accepting a change only if it improved validation AUC.

After several rounds the procedure converged to approximately **65% accuracy** and the AUC improvement per step dropped to near zero. The approach was abandoned for two reasons: (1) final performance was well below the standard gradient-boosted models; and (2) the random search made convergence slow and non-reproducible across different random seeds.

6.3 Feature ablation

A systematic feature removal and re-addition experiment was run on the final models. The largest change from removing any single feature was $\approx \pm 0.001$ AUC, confirming no single column dominates the signal. The only actionable outcome was the removal of one redundant feature that consistently produced a small stable improvement.

7 Conclusions

A principled cleaning pipeline reduced a 15M-row dataset to a high-quality 8M-row base; stratified sampling to 500k kept the project computationally feasible. Feature engineering — motivated by the observation that univariate signal is insufficient — added 17 financially

meaningful derived columns. Among all explored approaches, **CatBoost trained on the full class-weighted dataset achieved the highest ROC-AUC** and is recommended as the production baseline. Gradient-boosted trees outperformed all six deep-learning architectures at a fraction of the training cost, consistent with the broader tabular-data literature.